# AnotAI — PRD, Tech Spec e Guia do Time

Nov 5, 2026 · @Moisés Silveira de Oliveira

## 1. Visão geral

O AnotAI é um app de ditado por voz para Windows que escreve o que você fala em qualquer campo de texto onde o cursor estiver, já corrigido e pontuado em português, e traduz fala em inglês para português. Ele roda discreto no systray e é acionado por **Ctrl + Win**.

**Problema.** Digitar é lento (≈40 palavras/min) comparado a falar (≈130–150 palavras/min). O ditado nativo do Windows (Win + H) erra pontuação, não aplica norma culta e não traduz. Ferramentas de transcrição exigem copiar e colar entre janelas.

**Proposta de valor.** Segure Ctrl + Win, fale, solte: o texto aparece onde o cursor está — e-mail, Word, Teams, navegador, IDE, ERP — revisado segundo a norma culta e o Acordo Ortográfico vigente.

**Público inicial.** Profissionais que escrevem muito no Windows: analistas, suporte, gestores, advogados, médicos, professores e pessoas com limitação de digitação (acessibilidade).

**Métricas de sucesso (MVP)**

| Métrica | Meta MVP | Como medir |
|---|---|---|
| WER (taxa de erro de palavra) em pt-BR | ⩽ 6% em fala limpa | Benchmark interno de áudios rotulados |
| Latência soltar-tecla → texto inserido | ⩽ 1,2 s para frases de até 15 s (nuvem); ⩽ 2,5 s (local, GPU) | Telemetria opt-in |
| Inserção bem-sucedida no campo focado | ⩾ 98% nos 20 apps-alvo | Matriz de compatibilidade (seção 7) |
| Retenção semana 4 | ⩾ 40% dos usuários ativos | Telemetria opt-in |
| Palavras ditadas por usuário ativo/dia | ⩾ 500 | Telemetria opt-in |

**Nota sobre "padrão ABNT".** A ABNT normatiza a formatação de documentos (NBR 14724, 6023 etc.), não a ortografia. Para o ditado, o requisito foi interpretado como **norma culta + Acordo Ortográfico de 1990 (VOLP/ABL)**: ortografia, acentuação, pontuação, maiúsculas e concordância. Formatações ABNT (citações, referências) ficam como modo opcional futuro.

# 2. PRD — Requisitos

O MVP entrega ditado com correção e tradução EN→PT no Windows 10 (22H2) e 11, com dois modos de ativação por Ctrl + Win.

## 2.1 Requisitos funcionais

| ID | Requisito | Prioridade | Critério de aceite |
|---|---|---|---|
| RF-01 | **Push-to-talk:** segurar Ctrl + Win inicia a escuta; soltar encerra e insere o texto | P0 | Escuta começa em ⩽ 150 ms; o menu Iniciar não abre ao soltar |
| RF-02 | **Modo contínuo (toggle):** dois toques rápidos em Ctrl + Win (intervalo ⩽ 350 ms, configurável) mantêm a escuta até novo toque duplo ou simples | P0 | Indicador visual fixo enquanto ativo; texto inserido em blocos a cada pausa (VAD) |
| RF-03 | **Inserção no cursor:** o texto é escrito no campo com foco em qualquer app | P0 | Funciona nos 20 apps-alvo; clipboard do usuário restaurado após a inserção |
| RF-04 | **Detecção de campo editável:** se não houver campo de texto focado, o texto vai para o histórico e um aviso "Copiado — Ctrl+V para colar" aparece | P0 | Nenhum texto perdido |
| RF-05 | **Transcrição pt-BR de alta precisão** com pontuação automática | P0 | WER ⩽ 6% no benchmark |
| RF-06 | **Correção ortográfica e gramatical** (norma culta, Acordo de 1990), remoção de hesitações ("éé", "tipo", "né") e repetições | P0 | Liga/desliga; modo "literal" disponível |

| ID | Requisito | Prioridade | Critério de aceite |
|---|---|---|---|
| RF-07 | **Comandos de voz:** "vírgula", "ponto final", "nova linha", "novo parágrafo", "abre aspas", "apaga isso" | P1 | Lista visível nas configurações |
| RF-08 | **Tradução EN→PT:** com o modo Tradução ativo, fala em inglês é inserida em português | P0 | Alternar modo pelo tray, pela tela inicial ou por atalho Ctrl + Win + T |
| RF-09 | **Traduzir texto selecionado** (EN→PT) via atalho Ctrl + Win + Shift | P2 | Substitui a seleção |
| RF-10 | **Systray:** app minimiza para o tray ao fechar a janela; menu com Abrir, Modo (Ditado/Tradução), Pausar atalhos, Microfone, Sair | P0 | Ícone muda de estado (ocioso/escutando/processando/erro) |
| RF-11 | **Overlay flutuante:** pílula com forma de onda perto da base da tela durante a escuta | P0 | Sempre no topo, não rouba foco, clicável para cancelar |
| RF-12 | **Tela inicial:** status, modo ativo, últimas transcrições, estatísticas e atalhos | P0 | Ver seção 3 e mockup |
| RF-13 | **Histórico local** pesquisável, com copiar, reinserir e excluir | P1 | Retenção configurável (padrão 30 dias) |
| RF-14 | **Dicionário pessoal:** nomes, siglas e jargões (ex.: "eSales", "EDI", "MFT") | P1 | Termos aplicados como dica ao motor de transcrição e à correção |
| RF-15 | **Seleção de microfone** e teste de nível | P0 | Troca a quente, sem reiniciar |
| RF-16 | **Iniciar com o Windows** e atualização automática | P1 | Desativável |
| RF-17 | **Motor local (offline)** além do motor em nuvem | P1 | Usuário escolhe: Nuvem, Local ou Automático |

| ID | Requisito | Prioridade | Critério de aceite |
|---|---|---|---|
| RF-18 | **Onboarding** de 4 passos: microfone, atalho, teste guiado, privacidade | P0 | Concluído em < 2 min |

### 2.2 Requisitos não-funcionais

- **Desempenho:** app ocioso ⩽ 80 MB RAM e ~0% CPU; instalador ⩽ 15 MB (sem modelo local); modelo local baixado sob demanda.
- **Latência:** ver metas na seção 1; overlay aparece em ⩽ 100 ms.
- **Confiabilidade:** nenhum áudio perdido se a rede cair — fallback automático para o motor local ou fila de reenvio.
- **Privacidade:** áudio não é gravado em disco por padrão; LGPD (seção 6).
- **Acessibilidade:** navegação 100% por teclado, leitores de tela (Narrator/NVDA), contraste WCAG 2.2 AA, tamanhos de fonte escaláveis.
- **Idioma da UI:** pt-BR no MVP, i18n preparado para en e es.
- **Compatibilidade:** Windows 10 22H2+ e Windows 11, x64 e ARM64.

### 2.3 Fora do escopo do MVP

- Linux e Android (roadmap, seção 8).
- Outros pares de tradução além de EN→PT.
- Formatação ABNT de documentos (citações, referências).
- Contas, sincronização em nuvem e planos pagos (o backend de licenças entra na v1.1).
- Identificação de múltiplos falantes.

## 3. Experiência do usuário

A regra de UX é "zero janelas no caminho": o usuário nunca sai do app onde está escrevendo; todo feedback vem do overlay e do ícone do tray.

### 3.1 Fluxos principais

1. **Push-to-talk:** foco num campo → segura Ctrl + Win → overlay aparece pulsando → fala → solta → overlay mostra "Processando" → texto corrigido é inserido → overlay some em 600 ms.
2. **Modo contínuo:** toque duplo em Ctrl + Win → overlay trava com cadeado → a cada pausa de ~0,8 s o trecho é transcrito e inserido → novo toque encerra.

3. **Tradução:** usuário ativa o modo Tradução (tray, tela inicial ou Ctrl + Win + T) → overlay ganha o selo "EN→PT" → fala em inglês → texto em português é inserido.
4. **Sem campo editável:** texto vai para o clipboard e histórico; toast "Copiado — Ctrl+V para colar".
5. **Cancelar:** Esc durante a escuta ou clique no X do overlay descarta o áudio.

## 3.2 Estados do overlay e do ícone do tray

| Estado | Overlay | Ícone do tray | Cor |
|---|---|---|---|
| Ocioso | Oculto | Logo monocromático | Neutro |
| Escutando (segurando) | Pílula com ondas animadas pela voz | Ponto vermelho pulsante | Coral `#FF5A5F` |
| Escutando (contínuo) | Pílula + ícone de cadeado + cronômetro | Ponto vermelho fixo | Coral `#FF5A5F` |
| Processando | Barras "respirando" + texto "Escrevendo…" | Spinner | Violeta `#7C5CFF` |
| Tradução ativa | Selo "EN→PT" na pílula | Selo no ícone | Ciano `#22D3EE` |
| Erro / sem microfone | Mensagem curta + ação "Resolver" | Ponto âmbar | Âmbar `#FFB020` |

## 3.3 Tela inicial (janela principal)

Janela de 1100 × 720 px, cantos arredondados, material **Mica** do Windows 11 (fallback acrílico/sólido no Windows 10), tema claro/escuro automático.

- **Barra lateral:** Início, Histórico, Tradução, Dicionário, Configurações; rodapé com status do motor (Nuvem/Local).
- **Cartão-herói:** botão grande de microfone, modo atual (Ditado / Tradução EN→PT) em seletor segmentado, dica do atalho com teclas desenhadas.
- **Estatísticas:** palavras hoje, tempo economizado, precisão estimada, sequência de dias.
- **Recentes:** últimas 5 transcrições com app de destino, copiar e reinserir.
- **Atalhos:** cartão com Ctrl + Win (segurar), Ctrl + Win ×2 (contínuo), Ctrl + Win + T (tradução), Esc (cancelar).

Logo, cores, tipografia, mockup da tela inicial, estados do overlay e menu do systray: AnotAI — Identidade Visual e UI. Paleta principal: Violeta `#6D4AFF` (marca), Coral `#FF5A5F` (gravando), Ciano `#18C8E0` (tradução), Noite `#0E0B1F` (fundo escuro); fontes Sora (títulos), Inter (interface) e JetBrains Mono (atalhos).

### 3.4 Menu do systray

Abrir AnotAI · Modo: ● Ditado / ○ Tradução EN→PT · Microfone ▸ · Pausar atalhos · Histórico · Configurações · Sair. Clique simples no ícone abre a janela; clique direito abre o menu.

## 4. Stack recomendada

A recomendação é **Tauri 2 (núcleo em Rust) + React/TypeScript**: um único código gera Windows, Linux e Android, o instalador fica ~10 MB (contra ~100 MB do Electron) e o Rust dá acesso direto às APIs Win32 necessárias para hook de teclado, áudio e inserção de texto.

| Camada | Escolha | Por quê | Alternativa considerada |
|---|---|---|---|
| Shell desktop/mobile | **Tauri 2** | Binário pequeno, baixo consumo de RAM, suporte oficial a Windows, Linux, macOS, Android e iOS | Electron (pesado), .NET MAUI/WinUI 3 (sem Linux), Flutter (acesso nativo mais trabalhoso) |
| Núcleo nativo | **Rust** + crate `windows` (windows-rs) | Hook de teclado de baixo nível, SendInput, UI Automation, segurança de memória | C++ (mais risco), C# (não porta para Linux/Android com Tauri) |
| Front-end | **React 19 + TypeScript + Vite** | Ecossistema maior, fácil contratar | Svelte 5, SolidJS |
| Estilo | **Tailwind CSS v4** | Tokens de design como variáveis CSS, tema claro/escuro | CSS Modules |

| Camada | Escolha | Por quê | Alternativa considerada |
|---|---|---|---|
| Componentes | **shadcn/ui** (sobre Radix UI) | Acessível por padrão, código dentro do repo, fácil de aplicar a identidade AnotAI | Fluent UI React (visual Microsoft puro) |
| Animações | **Motion** (antigo Framer Motion) | Overlay, transições e microinterações fluidas | CSS puro |
| Ícones | **Lucide** | Traço fino, combina com a marca | Phosphor |
| Estado / dados | **Zustand** (UI) + **TanStack Query** (chamadas ao núcleo) | Simples, sem boilerplate | Redux Toolkit |
| Rotas | **TanStack Router** | Tipado de ponta a ponta | React Router |
| Captura de áudio | `cpal` (Rust) → 16 kHz mono | WASAPI no Windows, ALSA/PipeWire no Linux, AAudio no Android | Web Audio no front (não funciona com a janela oculta) |
| Detecção de voz (VAD) | **Silero VAD** via ONNX Runtime ( `ort` ) | Corta silêncio, segmenta o modo contínuo | WebRTC VAD (menos preciso) |
| Transcrição — nuvem | Interface `SttProvider` com adaptadores: **OpenAI gpt-4o-transcribe**, **Deepgram Nova-3**, **Groq Whisper large-v3-turbo** | Trocar de fornecedor sem mexer no app; escolha final pelo benchmark de WER/custo (seção 9) | Azure Speech |
| Transcrição — local | **whisper.cpp** via `whisper-rs` , modelo large-v3-turbo quantizado, aceleração Vulkan/CUDA | Offline e privado; fallback quando a rede cai | faster-whisper (exige Python embarcado) |

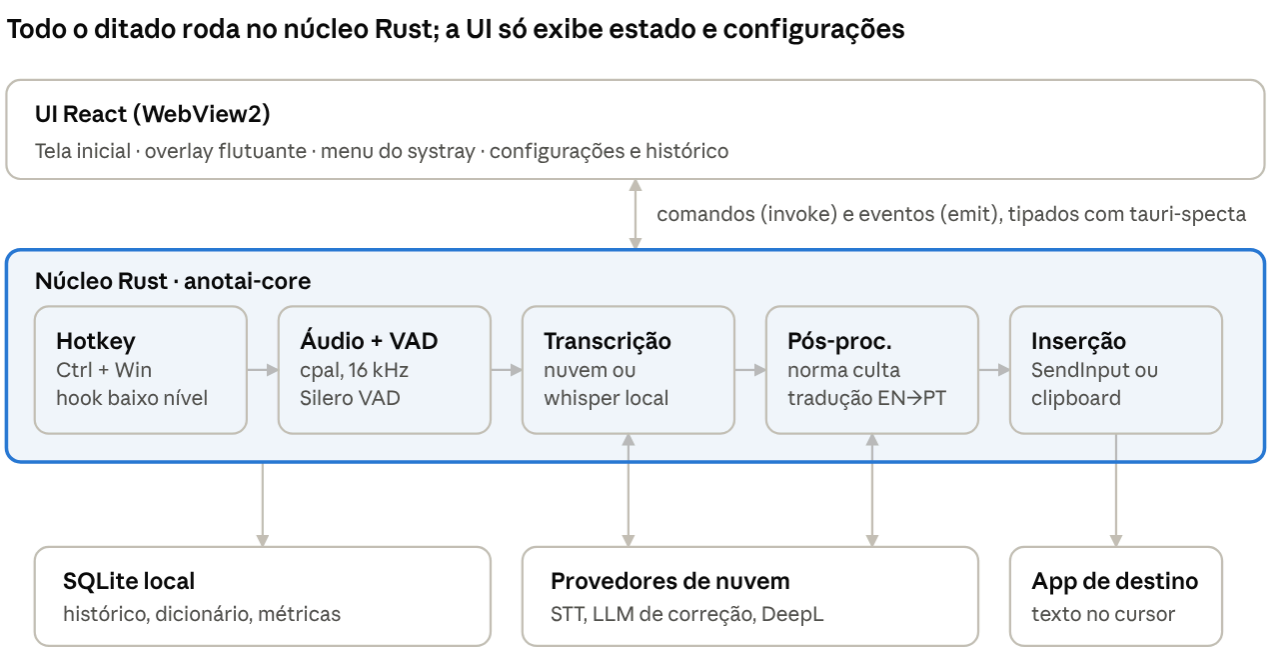
| Camada | Escolha | Por quê | Alternativa considerada |
|---|---|---|---|
| Correção / norma culta | **LLM rápido** (ex.: Claude Haiku) com prompt de revisão + dicionário pessoal; regras determinísticas antes (comandos de voz, hesitações) | Corrige pontuação, concordância e acentuação com contexto | LanguageTool (bom como 2ª camada offline) |
| Tradução EN→PT | **DeepL API** ou LLM (mesma chamada da correção) | Qualidade alta em pt-BR; uma chamada só traduz e revisa | Google Translate API; local: Opus-MT via CTranslate2 (v1.1) |
| Persistência | **SQLite** via `sqlx` + `tauri-plugin-store` (preferências) | Histórico com busca full-text (FTS5) | IndexedDB |
| Segredos | `keyring` (Credential Manager do Windows) | Chaves de API nunca em texto puro | Arquivo criptografado |
| Plugins Tauri | `tray-icon`, `autostart`, `single-instance`, `updater`, `notification`, `log` | Recursos prontos e mantidos | — |
| Backend (v1.1) | **Proxy de API** em Rust (Axum) ou Cloudflare Workers + Postgres | Esconde chaves dos provedores, controla cota e licenças | Chamada direta do cliente (só no MVP interno) |
| Testes | Vitest + Testing Library, `cargo test`, Playwright/WebdriverIO com `tauri-driver` | Cobre UI, núcleo e ponta a ponta | Jest |
| Qualidade | Biome (lint+format TS), `clippy` + `rustfmt`, Lefthook (git hooks), Conventional Commits | Rápido e padronizado | ESLint + Prettier |
| Monorepo | **pnpm workspaces** + Turborepo; Cargo workspace | Compartilha UI e núcleo entre desktop e mobile | Nx |

| Camada | Escolha | Por quê | Alternativa considerada |
|---|---|---|---|
| CI/CD | GitHub Actions ( `windows-latest` , `ubuntu-latest` ) + `tauri-action` | Build, assinatura e release automáticos | Azure DevOps |
| Assinatura de código | **Azure Trusted Signing** (ou certificado EV) | Evita o alerta do SmartScreen | Certificado OV comum |
| Distribuição | Instalador **NSIS/MSI** + Microsoft Store (MSIX) na v1.1 + `winget` | Canais corporativos e varejo | — |
| Observabilidade | Sentry (crashes Rust e JS) + PostHog (telemetria opt-in) | Diagnóstico com consentimento | — |

As versões exatas dos modelos de IA e SDKs devem ser confirmadas no início da Sprint 0, pois mudam com frequência.

## 5. Tech Spec — Arquitetura

O app tem um **núcleo Rust** ( `anotai-core` ) que faz tudo que é crítico em tempo real — hotkey, áudio, transcrição e inserção — e uma **UI React** que só exibe estado e configurações. Assim o ditado funciona mesmo com a janela fechada, e o núcleo pode ser reaproveitado no Linux e no Android.




arquitetura · núcleo Rust, UI, nuvem e armazenamento

O áudio percorre os cinco módulos da esquerda para a direita; a UI conversa com o núcleo só por comandos e eventos.

## 5.1 Módulos do núcleo ( `crates/` )

| Crate | Responsabilidade | APIs principais |
|---|---|---|
| `anotai-hotkey` | Detecta Ctrl + Win (segurar / toque duplo) e suprime o menu Iniciar | `SetWindowsHookExW(WH_KEYBOARD_LL)` |
| `anotai-audio` | Captura, reamostra para 16 kHz mono, buffer circular, nível do microfone | `cpal` , `rubato` |
| `anotai-vad` | Corta silêncio e segmenta o modo contínuo | Silero VAD via `ort` |
| `anotai-stt` | Trait `SttProvider` + adaptadores nuvem e local | `reqwest` , WebSocket, `whisper-rs` |
| `anotai-postproc` | Comandos de voz, hesitações, dicionário, correção por LLM, tradução | Trait `TextProcessor` |
| `anotai-inject` | Detecta campo editável e insere o texto | UI Automation, `SendInput` , clipboard |

| Crate | Responsabilidade | APIs principais |
|---|---|---|
| `anotai-store` | Histórico, dicionário, métricas | SQLite (`sqlx`), FTS5 |
| `anotai-platform` | Abstração por SO: `windows/`, `linux/`, `android/` | `#[cfg(target_os)]` |

## 5.2 Pipeline áudio → texto → inserção

1. **Hotkey down** → o núcleo grava o `HWND` e o elemento focado (UI Automation) *antes* de mostrar o overlay, para devolver o texto ao lugar certo.
2. **Captura** começa com pré-buffer de 300 ms (não perde a primeira sílaba).
3. **Streaming:** os quadros vão por WebSocket ao provedor de nuvem enquanto o usuário fala (transcrição parcial reduz a latência final). No motor local, o áudio é processado ao soltar a tecla.
4. **Hotkey up / pausa do VAD** → transcrição final.
5. **Pós-processamento** em cadeia: comandos de voz → limpeza de hesitações → dicionário → LLM (correção **ou** tradução + correção numa única chamada, com timeout de 1,5 s; se estourar, insere o texto bruto já pontuado).
6. **Inserção:** reativa a janela de origem se o foco mudou; até 200 caracteres usa `SendInput` com `KEYEVENTF_UNICODE`; acima disso, salva o clipboard, cola com Ctrl+V e restaura o clipboard 300 ms depois.
7. **Registro** no histórico e evento para a UI atualizar estatísticas.

## 5.3 Hotkey Ctrl + Win — detalhes críticos

- `RegisterHotKey` **não** aceita combinação só de modificadores; por isso usamos hook de baixo nível (`WH_KEYBOARD_LL`) numa thread dedicada com loop de mensagens.
- **Menu Iniciar:** ao soltar a tecla Win o Windows abre o Iniciar. O hook injeta uma tecla "máscara" neutra (VK `0xE8`, não atribuída) antes do key-up da Win — a mesma técnica usada pelo AutoHotkey.
- **Conflitos:** Ctrl + Win + ←/→/D/F4 são atalhos de áreas de trabalho virtuais. Se outra tecla for pressionada com Ctrl + Win, o AnotAI cancela a escuta e deixa o atalho do Windows passar.
- **Segurar vs. toque duplo:** toque = pressionar e soltar em < 250 ms; dois toques em ⩽ 350 ms = modo contínuo; segurar > 250 ms = push-to-talk. Valores configuráveis.
- **Desempenho:** o callback do hook precisa responder em < 1 ms (o Windows remove hooks lentos); toda lógica vai para um canal (`crossbeam`) consumido por outra thread.

- **Atalho alternativo** configurável (ex.: Ctrl + Alt + Espaço) para teclados sem tecla Win e para ambientes corporativos que bloqueiam hooks.
- **Apps elevados:** se o app de destino roda como administrador, o hook e o `SendInput` não alcançam (UIPI). Mostrar aviso; oferecer opção "Executar como administrador".

### 5.4 Comunicação núcleo ↔ UI

Comandos Tauri ( `invoke` ) tipados com **tauri-specta**, que gera os tipos TypeScript a partir do Rust. Eventos do núcleo ( `state-changed` , `audio-level` , `transcript-partial` , `transcript-final` ) chegam à UI via `emit` . O overlay é uma segunda janela Tauri transparente, sem borda, `always_on_top` , `focusable: false` e `skip_taskbar` .

## 6. Dados, configurações, segurança e LGPD

Todos os dados ficam na máquina do usuário em `%APPDATA%\AnotAI\` ; o áudio só sai do computador quando o motor de nuvem está ativo e nunca é gravado em disco por padrão.

### 6.1 Modelo de dados (SQLite)

```
CREATE TABLE transcription (
  id            TEXT PRIMARY KEY,          -- UUID v7
  created_at    INTEGER NOT NULL,          -- epoch ms
  mode          TEXT NOT NULL,             -- 'dictation' | 'translation'
  trigger       TEXT NOT NULL,             -- 'hold' | 'toggle'
  raw_text      TEXT NOT NULL,             -- saída do STT
  final_text    TEXT NOT NULL,             -- após correção/tradução
  source_lang   TEXT NOT NULL,             -- 'pt-BR' | 'en'
  target_lang   TEXT NOT NULL,
  target_app    TEXT,                      -- ex.: 'OUTLOOK.EXE'
  inserted      INTEGER NOT NULL,          -- 1 = inserido no cursor
  duration_ms   INTEGER NOT NULL,
  latency_ms    INTEGER NOT NULL,
  engine        TEXT NOT NULL              -- 'cloud:deepgram' |
'local:whisper'
);
CREATE VIRTUAL TABLE transcription_fts USING fts5(final_text,
content='transcription');

CREATE TABLE dictionary_term (
  id          TEXT PRIMARY KEY,
  term        TEXT NOT NULL UNIQUE,        -- ex.: 'eSales'
  spoken_as   TEXT,                        -- ex.: 'i sales'
  created_at  INTEGER NOT NULL
);
```

```
CREATE TABLE daily_stat (
  day         TEXT PRIMARY KEY,            -- 'YYYY-MM-DD'
  words       INTEGER NOT NULL DEFAULT 0,
  sessions    INTEGER NOT NULL DEFAULT 0,
  seconds     INTEGER NOT NULL DEFAULT 0
);
```

Migrações versionadas com `sqlx migrate`, executadas na inicialização.

## 6.2 Configurações (`settings.json` via tauri-plugin-store)

| Chave | Padrão | Descrição |
|---|---|---|
| `hotkey.primary` | `Ctrl+Win` | Atalho principal |
| `hotkey.doubleTapMs` | `350` | Janela do toque duplo |
| `hotkey.holdThresholdMs` | `250` | A partir daqui conta como "segurar" |
| `engine.mode` | `auto` | `cloud`, `local` ou `auto` (nuvem com fallback local) |
| `engine.cloudProvider` | definido pelo benchmark | Provedor STT |
| `postproc.correction` | `true` | Correção norma culta |
| `postproc.removeFillers` | `true` | Remove "éé", "tipo", "né" |
| `mode.current` | `dictation` | `dictation` ou `translation` |
| `insert.method` | `auto` | `auto`, `type` ou `paste` |
| `history.retentionDays` | `30` | 0 = não guardar |
| `audio.saveRecordings` | `false` | Guardar áudio (debug) |
| `ui.theme` | `system` | `light`, `dark`, `system` |
| `app.launchAtStartup` | `true` | Iniciar com o Windows |
| `telemetry.enabled` | `false` | Opt-in no onboarding |

## 6.3 Segurança

- Chaves de API no **Windows Credential Manager** (`keyring`); no build público, o cliente fala só com o proxy AnotAI (v1.1), nunca direto com os provedores.
- TLS 1.2+ com pinning do domínio do proxy.

- Tauri com **capabilities** mínimas por janela (o overlay não tem acesso a sistema de arquivos) e CSP restrita.
- O hook de teclado **só observa** Ctrl, Win e as teclas pressionadas junto com eles; nenhuma outra tecla é registrada ou logada. Isso precisa constar na política de privacidade e ser auditado no code review (antivírus tratam hooks como sinal de keylogger).
- Binário assinado; atualizações assinadas (chave do `tauri-plugin-updater`).
- Campos de senha (UI Automation `IsPassword = true`) **bloqueiam** a inserção.

### 6.4 LGPD

- Base legal: consentimento explícito no onboarding para processamento de voz na nuvem.
- Contratos (DPA) com provedores que garantam **não retenção** e **não uso para treino**; preferir regiões com adequação ou cláusulas-padrão.
- Direitos do titular dentro do app: exportar histórico (JSON/CSV) e "Apagar tudo".
- Modo **Local** como opção para clientes corporativos com dados sensíveis.
- Telemetria sem conteúdo: nunca envia texto ou áudio, só métricas agregadas.

## 7. Repositório, padrões, testes e CI/CD

Um monorepo único (`anotai/`) guarda desktop, mobile, UI compartilhada e o núcleo Rust, para que Linux e Android reutilizem o mesmo código.

## 7.1 Estrutura

```
anotai/
├─ apps/
|  ├─ desktop/            # Tauri 2: src/ (React) + src-tauri/ (Rust,
janelas, tray)
|  └─ mobile/             # v2: Tauri Android + módulo IME em Kotlin
├─ packages/
|  ├─ ui/                 # componentes shadcn/ui com a identidade AnotAI
|  ├─ design-tokens/      # cores, tipografia, raios, sombras (CSS vars + TS)
|  └─ i18n/               # textos pt-BR / en / es
├─ crates/
|  ├─ anotai-core/        # orquestra o pipeline
|  ├─ anotai-hotkey/  anotai-audio/  anotai-vad/  anotai-stt/
|  ├─ anotai-postproc/  anotai-inject/  anotai-store/  anotai-platform/
├─ services/
|  └─ api-proxy/          # v1.1: Axum, cotas, licenças
├─ tools/benchmark/       # áudios rotulados + script de WER
├─ docs/                  # ADRs, PRD, runbooks
├─ .github/workflows/
├─ Cargo.toml  pnpm-workspace.yaml  turbo.json  biome.json  lefthook.yml
```

## 7.2 Ambiente de desenvolvimento

1. Windows 11 com **Visual Studio Build Tools 2022** (workload "Desktop development with C++") e WebView2 (já vem no Windows 11).
2. Rust stable via `rustup` ( `x86_64-pc-windows-msvc` ) e Node.js LTS + `pnpm` (via Corepack).
3. Opcional para motor local com GPU: Vulkan SDK ou CUDA Toolkit.
4. `pnpm install` → `pnpm dev` (abre o app com hot reload) → `pnpm test` .
5. Chaves de provedores em `.env.local` (não versionado) apenas em desenvolvimento.

## 7.3 Padrões

- **Git:** trunk-based, branches curtas `feat/` , `fix/` , `chore/` ; PR com 1 aprovação e CI verde; Conventional Commits; versionamento com Changesets.
- **Decisões de arquitetura** registradas como ADR em `docs/adr/` (primeiras: ADR-001 Tauri, ADR-002 hook de teclado, ADR-003 provedor STT).
- **TypeScript** em `strict` ; sem `any` ; componentes em PascalCase; hooks `useX` .
- **Rust:** `clippy -D warnings` , erros com `thiserror` (libs) e `anyhow` (app), logs com `tracing` ; nada de `unwrap()` fora de testes.

- **UI:** só tokens do `design-tokens` (nada de cor solta); todo componente com estado de foco visível e rótulo acessível.
- **Definition of Done:** testes, docs atualizadas, acessibilidade verificada, testado na matriz de apps.

### 7.4 Estratégia de testes

| Tipo | Ferramenta | Cobertura mínima |
|---|---|---|
| Unitário Rust | `cargo test`, `insta` (snapshots do pós-processamento) | 80% em `postproc`, `hotkey` (máquina de estados) |
| Unitário UI | Vitest + Testing Library | 70% |
| Ponta a ponta | WebdriverIO + `tauri-driver` no Windows | Fluxos do onboarding, tela inicial, configurações |
| Inserção real | Script que abre Notepad, Word, Chrome, Teams e verifica o texto | Os 20 apps-alvo |
| Qualidade de transcrição | `tools/benchmark`: WER e latência por provedor | Roda a cada mudança em `stt` / `postproc` |

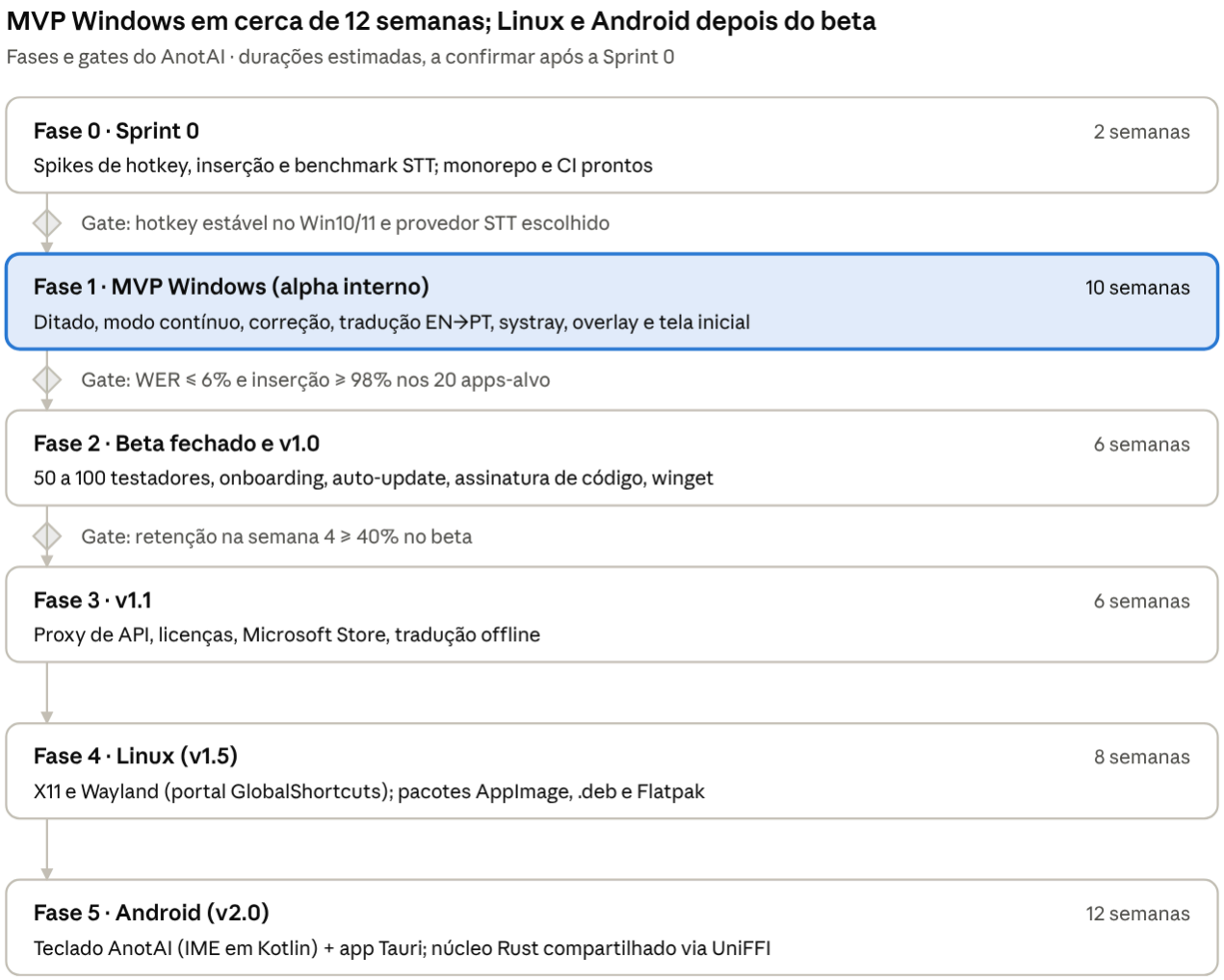
**Matriz de 20 apps-alvo:** Notepad, Word, Excel, Outlook (novo e clássico), Teams, Chrome, Edge, Firefox, Gmail web, WhatsApp Desktop, Slack, VS Code, Visual Studio, OneNote, PowerPoint, Explorador de Arquivos (renomear), LibreOffice Writer, Discord, SAP GUI, Terminal.

### 7.5 CI/CD

- **PR:** lint (Biome, clippy), testes, build de verificação em `windows-latest`.
- **main:** build noturno assinado publicado no canal *beta* do updater.
- **Tag `v*`:** `tauri-action` gera instaladores x64 e ARM64, assina via Azure Trusted Signing, publica release no GitHub e o manifesto do updater; depois envia ao `winget`.
- **Canais:** `beta` (time e testadores) e `stable`.

## 8. Roadmap

O MVP para Windows chega em cerca de 12 semanas (Sprint 0 + 5 sprints de 2 semanas); Linux e Android só começam depois que o beta comprovar retenção.




roadmap · 6 fases, 3 gates

Cada gate precisa ser cumprido para a fase seguinte começar; o total estimado até o Android é de cerca de 44 semanas com um time de 3 a 4 pessoas (2 Rust/desktop, 1 front-end, 1 QA/DevOps compartilhado).

# 9. Riscos, decisões em aberto e próximos passos

O maior risco técnico é o atalho Ctrl + Win (hook de teclado + supressão do menu Iniciar); por isso ele é o primeiro spike da Sprint 0.

## 9.1 Riscos

| Risco | Impacto | Mitigação |
|---|---|---|
| Antivírus/EDR corporativo marca o hook de teclado como keylogger | Alto | Binário assinado, submissão prévia à Microsoft Defender, documentação para TI, atalho alternativo |

| Risco | Impacto | Mitigação |
|---|---|---|
| Menu Iniciar abre ao soltar Win em alguma build do Windows | Médio | Tecla-máscara + testes automatizados em Win10 e Win11 (23H2, 24H2, 25H2) |
| Inserção falha em apps específicos (Electron, Java, SAP, apps elevados) | Médio | Estratégias `type` / `paste` por app, matriz de compatibilidade, fallback para clipboard |
| Latência alta na correção por LLM | Médio | Streaming, timeout de 1,5 s com fallback ao texto bruto, modelo pequeno |
| Custo de API por usuário | Médio | Medir no benchmark; motor local como padrão para quem tem GPU; cotas no proxy |
| Android não permite "digitar em qualquer app" sem teclado próprio | Alto (v2) | Implementar como teclado (IME) em Kotlin chamando o núcleo Rust via UniFFI |
| Wayland no Linux bloqueia hooks globais e injeção de teclas | Alto (v1.5) | Portal `GlobalShortcuts` do XDG + `ydotool` /IBus; X11 com `XRecord` / `XTest` |

## 9.2 Decisões em aberto

- [ ] Provedor STT padrão (Deepgram, OpenAI ou Groq) — decidir pelo benchmark de WER pt-BR, latência e custo na Sprint 0.
- [ ] Tradução via DeepL ou via o mesmo LLM da correção.
- [ ] Modelo de negócio: gratuito com cota + plano Pro? Licença corporativa?
- [ ] Produto interno da eSales ou produto comercial independente (afeta marca, contratos e LGPD).
- [ ] Nome de domínio e registro da marca "AnotAI" no INPI.

## 9.3 Próximos passos para o time

- [ ] Criar o monorepo com o template da seção 7.1 e o pipeline de CI.
- [ ] Spike 1: hook Ctrl + Win com supressão do Iniciar e máquina de estados segurar/toque duplo.
- [ ] Spike 2: inserção de texto nos 20 apps-alvo ( `SendInput` vs. clipboard).
- [ ] Spike 3: benchmark STT com 100 áudios pt-BR e 50 em inglês gravados pelo time.
- [ ] Implementar os tokens de design e o kit de componentes a partir do guia de marca.

- [ ] Escrever ADR-001 a ADR-003.